FinSight: Intelligent Credit Risk Analysis for Accurate and Explainable Loan Applicant Classification

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

M. Manasvini	**: 2420030475**
N. Sreeja	**: 2420030267**
M. Sahasra	**: 2420030459**
S. Sreeja	**: 2420030548**

Under the Esteemed Guidance of

Ms. Prathusha
Assistant Professor

Department of Computer Science and Engineering

AY 2026-2027

Semester: ODD Sem

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

We, M. Manasvini (2420030475), N. Sreeja (2420030267), M. Sahasra (2420030459), S. Sreeja (2420030548), declare that the project report entitled "FinSight: Intelligent Credit Risk Analysis for Accurate and Explainable Loan Applicant Classification" records our work undertaken in partial fulfillment of the requirements for the award of B.Tech in Computer Science and Engineering at Koneru Lakshmaiah Education Foundation, Hyderabad, India. We have acknowledged the datasets, literature, software and documentation used. The report accurately distinguishes implemented work, measured results and proposed enhancements.

M. Manasvini – 2420030475

N. Sreeja – 2420030267

M. Sahasra – 2420030459

S. Sreeja – 2420030548

Koneru Lakshmaiah Education Foundation
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled "FinSight: Intelligent Credit Risk Analysis for Accurate and Explainable Loan Applicant Classification" is a record of the work carried out and submitted by M. Manasvini (2420030475), N. Sreeja (2420030267), M. Sahasra (2420030459), S. Sreeja (2420030548), in partial fulfillment of the requirements for the award of BACHELOR OF TECHNOLOGY in Computer Science and Engineering at Koneru Lakshmaiah Education Foundation, Hyderabad, India, during the academic year 2026-2027, under the guidance of Ms. Prathusha, Associate Professor.

Guide signature:
Ms. Prathusha,
Assistant Professor.

Course Coordinator: Ms. Nalajala Sunanda
Signature:

HOD: Mr. Venkateswara Rao Pulipati
Signature:

ACKNOWLEDGEMENT

We express our sincere gratitude to Ms. Prathusha, Associate Professor and our Project Guide, for her guidance and academic support during the development and evaluation of FinSight.

We thank our Course Coordinator Ms. Nalajala Sunanda, for coordinating the project work, and our Head of Department, Mr.Venkateswara Rao Pulipati for the departmental support provided to us.

We acknowledge the Department of Computer Science and Engineering and Koneru Lakshmaiah Education Foundation, Hyderabad, India, for the learning environment and facilities available for our project. We also thank the teaching and non-teaching staff for their assistance.

We are grateful to our families, peers and classmates for their encouragement and cooperation. We acknowledge the public dataset maintainers, research authors and open-source software communities whose resources supported this work.

M. Manasvini – 2420030475

N. Sreeja – 2420030267

M. Sahasra – 2420030459

S. Sreeja – 2420030548

Abstract

FinSight is an educational credit-risk analytics and decision-support project that examines how loan applicants can be classified accurately while making model behavior understandable. Classification errors have asymmetric consequences: missing a bad-risk applicant can create repayment exposure, whereas incorrectly flagging a good-risk applicant increases review burden. The project uses the UCI Statlog German Credit dataset containing 1,000 applicants, 20 predictors, and a 700:300 good-to-bad target distribution. Schema checks, explicit target mapping, and a stratified 800/200 training–test split establish the data foundation. Numeric imputation, optional standardization, and categorical one-hot encoding are fitted through reusable training pipelines.

Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting are compared using five-fold validation, predictive metrics, and an asymmetric cost of five times false negatives plus false positives. Decision thresholds are selected from out-of-fold training probabilities. Gradient Boosting is selected at threshold 0.13 because it has the lowest training cost. On the 200-applicant holdout, it achieves accuracy 0.5500, precision 0.3881, bad-risk recall 0.8667, F1-score 0.5361, ROC-AUC 0.8005, and cost 122. Logistic Regression achieves the lowest primary holdout cost, 107, demonstrating that the training winner does not necessarily remain the most cost-effective model on unseen data.

The saved pipeline produces a class-1 risk probability and a model-derived 0–100 educational risk score. SHAP provides global and applicant-level explanations. A Flask dashboard supports portfolio analytics, applicant assessment, model comparison and What-If analysis. Amazon S3, AWS Glue Data Catalog and Amazon Athena support the implemented cloud analytics path. Additional publication analyses quantify bootstrap uncertainty, paired model differences, threshold sensitivity and calibration. FinSight remains a human-reviewed academic prototype: historical benchmark limitations, false-positive trade-offs and explanation constraints prevent its use as an autonomous real-world loan approval system.

Index

INTRODUCTION 1
SYSTEM REQUIREMENTS 3
TECHNOLOGY STACK 4
SYSTEM ARCHITECTURE 5
DESIGN 7
IMPLEMENTATION 10
FEATURES 16
TESTING 18
DEPLOYMENT 20
CHALLENGES AND LIMITATIONS 22
FUTURE ENHANCEMENTS 24
CONCLUSION 25
REFERENCES 26
APPENDICES 28

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

Credit-risk analysis estimates how applicant information relates to historical good- and bad-risk outcomes. Accuracy alone is insufficient when the two classification errors have different costs. In the German Credit benchmark, an all-good classifier reaches 70% accuracy but misses every bad-risk applicant. FinSight makes this limitation visible through explicit positive-class metrics, asymmetric cost and explanations. Explainable financial modeling [1] and caution about high-stakes black boxes [2] motivate the design.

Related credit-scoring studies investigate explainable deep models [4], auditability [5], bank assessments [6] and integrated credit evaluation [7]. Work comparing SHAP and LIME [8] and reviewing financial XAI [9] supports treating explanation quality as an evaluation question. Threshold-learning research [12] further motivates connecting classification cutoffs to decision consequences. FinSight applies these established ideas within a reproducible educational benchmark.

1.2 Problem Statement

The project addresses the need for a reproducible credit-risk comparison that connects model predictions to operating thresholds and understandable applicant-level factors. A high ROC-AUC does not specify how many risky applicants will be missed at a chosen cutoff. The implementation therefore separates probability estimation, classification, educational score display and human interpretation. It also tests whether the lowest-cost model on training data retains that ranking on independent observations.

1.3 Scope of the Project

The implemented scope includes public-data acquisition, validation, exploratory analysis, four classifier families, training-derived thresholds, persisted inference, SHAP explanations, a local Flask dashboard and an AWS analytics path. The dashboard accepts the 20 original predictors and supports What-If comparisons. Authentication, production underwriting, payment systems, online retraining and institution-wide monitoring are outside the current implementation.

1.4 Objectives

Our objectives are to preserve a validated data source; prevent identifier and target leakage; compare LR, DT, RF and GB consistently; optimize thresholds under a 5:1 error penalty; expose model probability and an educational score; explain model behavior globally and locally; provide a usable dashboard; support S3/Glue/Athena analytics; and verify core behavior through automated tests.

1.5 Importance of the Application

FinSight helps demonstrate why a classification decision is more than a model name or an accuracy value. Its results show the operational trade-off between finding more bad-risk applicants and creating more false alarms. Reusing one saved pipeline also reduces inconsistency between experimentation and application inference. SHAP [3] gives reviewers a way to inspect contributions while keeping causal and fairness claims separate.

1.6 Target Users and Audience

The intended users are students, project evaluators, researchers reproducing the benchmark, and instructors demonstrating responsible machine-learning workflows. Analyst-like review interactions are simulated for educational purposes. The system is not offered as a service for actual borrowers or as an autonomous loan approval or rejection tool.

CHAPTER 2
SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Table 2.1 gives practical development guidance rather than measured minimum specifications. The benchmark is small; model training and SHAP generation use CPU resources, and no GPU dependency is present.

Table 2.1 Hardware requirements

Component	Practical requirement
Processor	Modern 64-bit, four-core CPU recommended
Memory	8 GB workable; 16 GB recommended for notebooks and repeated experiments
Storage	At least 2 GB free for environment, artifacts and reports
Display	1280 × 720 or higher; larger display useful for dashboard review
Connectivity	Internet for initial dependencies/data and optional AWS/CDN assets

2.2 Software Requirements

Table 2.2 Software requirements

Software	Use
Windows / PowerShell	Verified local execution environment
Python 3.10–3.12	Recommended project interpreter range
Virtual environment and pip	Isolated dependency installation
Modern browser	Local dashboard and interactive Plotly charts
Git	Version control and reproducible source management
AWS account and configured identity	Optional deployed S3, Glue catalog and Athena analytics
Editor / IDE	A Python-capable editor; the repository does not establish a required IDE

2.3 Development Tools and Frameworks

The repository uses Python modules for reusable logic, a notebook for exploratory narration, Flask/Jinja templates for the application, and Pytest for verification. Any suitable editor, including VS Code, can be used; no editor-specific project requirement is claimed. Initial dependency installation and external browser assets require connectivity, while the local application reads cached data and saved artifacts without live AWS requests.

CHAPTER 3
TECHNOLOGY STACK

Table 3.1 Technologies and their implemented roles

Technology	Role in FinSight
Python; Pandas; NumPy	Data loading, schema checks, analytics and numeric computation
Scikit-learn	Preprocessing pipelines, classification, validation, metrics and calibration
SHAP; Joblib	Model explanations and persisted model/pipeline artifacts
Flask; Jinja; HTML; CSS	Server routes, templates, forms and local presentation
Bootstrap; Plotly	Responsive interface components and interactive portfolio plots
Boto3; Amazon S3	Credential-resolved object upload, download and existence checks
AWS Glue Data Catalog; Athena	Schema registration and SQL over the S3 analytics CSV
Pytest; Git / GitHub	Automated verification, version control and optional repository sharing

3.1 Data and Machine-Learning Libraries

Pandas maintains the named tabular schema and exports the human-readable analytics CSV. NumPy supports probability, score and explanation calculations. Scikit-learn [15] keeps transformations and estimators together so the same feature handling is applied during training and inference. Matplotlib generates the saved evaluation figures. SHAP explains the selected fitted tree ensemble, and Joblib persists the complete inference contract.

3.2 Application Technologies

Flask [16] handles HTTP requests and server-side validation. Jinja renders HTML templates using actual project outputs. CSS and Bootstrap organize forms and cards; Plotly supports interactive descriptive charts. The front end does not train models or independently recompute their feature preprocessing. Six routes provide overview, portfolio, assessment, explainability, performance and What-If views.

3.3 Cloud Services and Version Control

Boto3 follows the AWS credential-provider chain rather than storing keys in source files. Amazon S3 holds analytics objects [17], Glue Data Catalog describes the external table [18], and Athena queries it with SQL [19]. Git supports source versioning; no publication-ready public GitHub URL is asserted. Neither a conventional relational application database nor a hosted Flask production service is implemented.

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 High-Level Architecture

Fig. 4.1 separates development-time data preparation and learning from application-time inference. Raw applicant data is validated before modeling. Training produces an artifact containing preprocessing, estimator, threshold and feature contract. The dashboard consumes this artifact and descriptive analytics files. Cloud analytics form a parallel reporting path rather than an inference dependency.

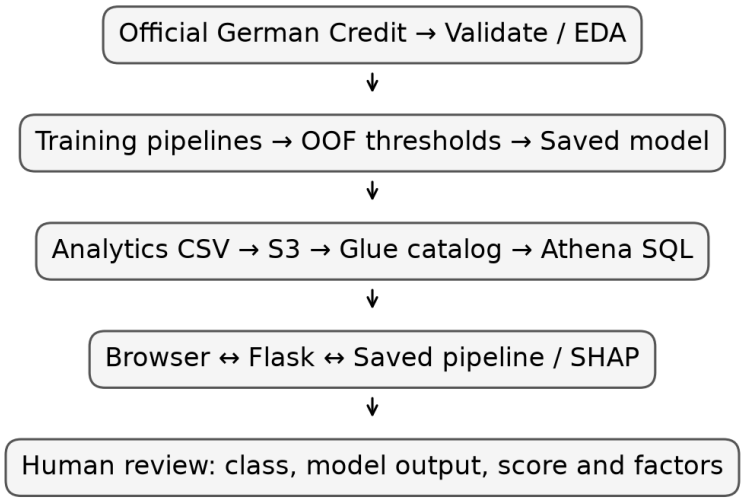


Fig. 4.1 FinSight high-level architecture

4.2 Data Pipeline

The data path is acquisition → pinned checksum and schema validation → descriptive EDA → raw-feature split → fitted preprocessing. A separate processed CSV adds applicant_id, a readable credit-risk label and is_bad_risk for analytics. These added columns are useful in SQL summaries but are excluded from the model input. Source data and provenance metadata remain separate from generated exports.

4.3 Machine-Learning Architecture

The training path wraps each classifier in a ColumnTransformer-based pipeline. Cross-validation and compact hyperparameter searches operate on the development partition. OOF probabilities select a cost-aware threshold for each model, then the final model is chosen from

training evidence. Independent holdout evaluation documents generalization without changing that choice. The selected pipeline and threshold are saved together.

4.4 Cloud Architecture

The implemented cloud path is Amazon S3 → AWS Glue Data Catalog → Amazon Athena → SQL analytics. The table was registered through Athena DDL because crawler-creation permission was unavailable. This is a valid catalog-registration alternative; the report does not claim a successful crawler deployment. Existing project records document verification in eu-north-1. No new live cloud operation was performed while preparing this report.

4.5 Flask Application Architecture

The Flask application loads saved artifacts through reusable functions and caches them per process. Routes parse allowed raw features, validate category and numeric constraints, call predict_applicant and render templates. Portfolio pages use the processed CSV, while performance pages use saved metrics and allow-listed figures. Inference is independent of model fitting, and What-If comparisons evaluate two profiles using the same pipeline.

CHAPTER 5
DESIGN

5.1 System Workflow

The workflow starts with a validated benchmark and ends with human review of model outputs. Development operations are reproducible commands rather than actions triggered by a web request. Inference requires the exact feature contract stored with the artifact. Classification threshold, probability, score band and explanation are displayed as distinct concepts.

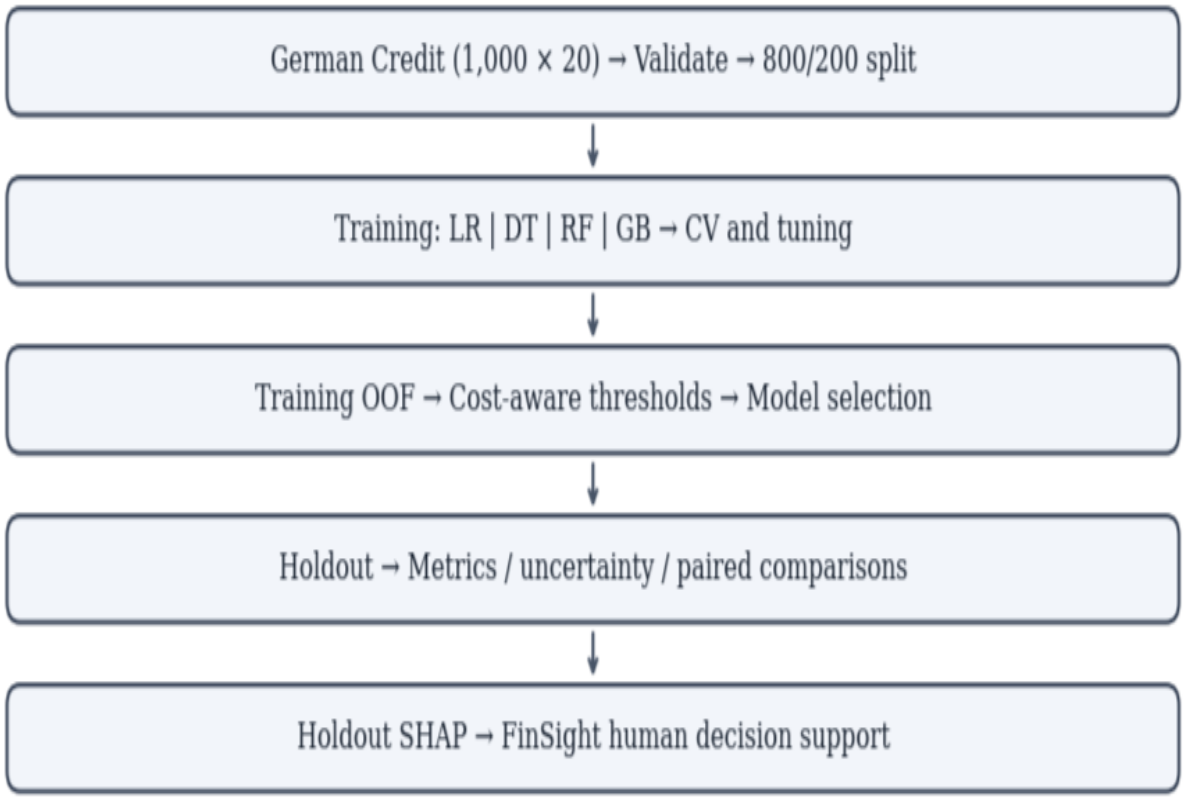


Fig. 5.1 Training, evaluation and explainability workflow

5.2 Data-Flow Diagram

Fig. 5.2 shows the major data exchanges. The external user provides raw predictor values; the application returns classification, model output, score and signed explanation factors. The model repository supplies an immutable fitted pipeline for the request. This design does not require an entity–relationship diagram because there is no conventional relational application database.

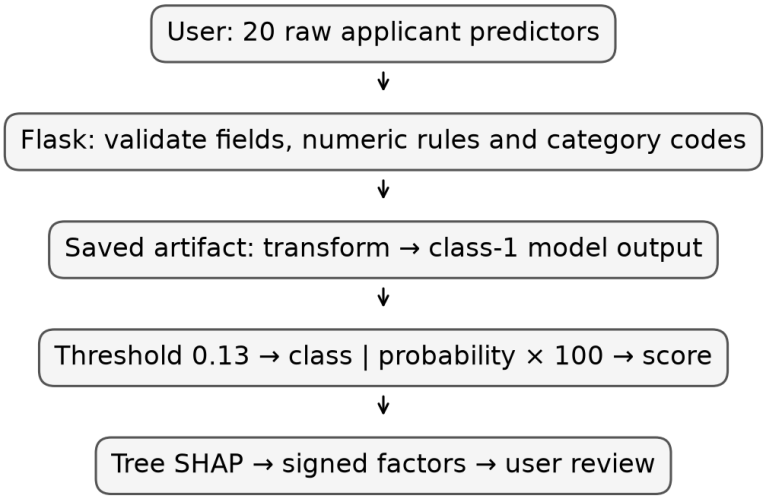


Fig. 5.2 Applicant assessment data-flow diagram

5.3 Dataset and Feature Schema

Table 5.1 Dataset summary

Property	**Verified value**
Dataset	UCI Statlog German Credit, original symbolic data [13]
Dimensions	1,000 rows; 20 predictors plus source target
Class distribution	700 good risk; 300 bad risk
Raw / model target	1 = good, 2 = bad / 0 = good, 1 = bad
Model split	800 development and 200 test; stratified; seed 42
Analytics export	1,000 rows; original columns plus three helper fields
Quality report	No missing values, duplicate rows or invalid values reported

Table 5.2 Predictor schema

Predictor group	**Fields**
Accounts and history	checking_account_status, credit_history, savings_status
Loan request	duration_months, purpose, credit_amount, installment_rate
Employment and resources	employment_duration, property, housing, job
Personal and household	personal_status_sex, age_years, dependents, foreign_worker
Other obligations / context	other_debtors, residence_duration, other_installment_plans, existing_credits, telephone

5.4 Machine-Learning Pipeline Design

The pipeline maps raw labels explicitly, splits before fitting transformations, imputes numeric values with medians, imputes categorical values with their most frequent category and applies one-hot encoding. Numeric standardization is enabled for LR and disabled for tree models. Unknown categories are ignored safely by the encoder. Applicant identifiers and target-derived helpers are rejected from feature inputs.

5.5 User Interface Design

Navigation separates portfolio description from individual assessment and model evaluation. Forms use readable category labels but submit the original category codes expected by the model. Inline messages handle malformed or incomplete inputs without exposing tracebacks. The interface distinguishes the 13% classification threshold from display bands at scores 30 and 60. A visible educational disclaimer accompanies the decision-support interpretation.

CHAPTER 6
IMPLEMENTATION

6.1 Dataset Acquisition

The acquisition module retrieves the official symbolic german.data file when no valid cache is present. It records retrieval metadata and a SHA-256 checksum. Cached and newly downloaded files are checked against the pinned official-file digest. Reusing the cache makes subsequent runs independent of network availability and preserves the original categorical codes.

6.2 Data Validation

The schema defines the ordered 21 columns and categorical meanings. Validation checks 1,000 rows, completeness, target codes, documented category values, and finite positive integer numeric fields. Exact duplicates are reported rather than automatically removed. The present quality report records zero missing values, duplicate rows and invalid values. These checks establish data-contract consistency, not exhaustive detection of every domain anomaly.

6.3 Data Preprocessing

split_features_target accepts the ordered raw schema and maps bad risk to class 1. make_train_test_split uses random_state=42 and target stratification. Each candidate model receives its own preprocessing pipeline; imputers, encoder categories and LR scaling parameters are learned only from training folds. The analytics export is never used as a model matrix because it includes target-derived columns.

6.4 Exploratory Data Analysis

EDA generates target, numeric and categorical distributions and feature–risk relationships. IQR flags describe unusual numeric observations without removing them. The dataset is 70% good and 30% bad; requested amounts and durations vary substantially. These figures support interpretation of the historical benchmark rather than causal claims about applicant behavior.

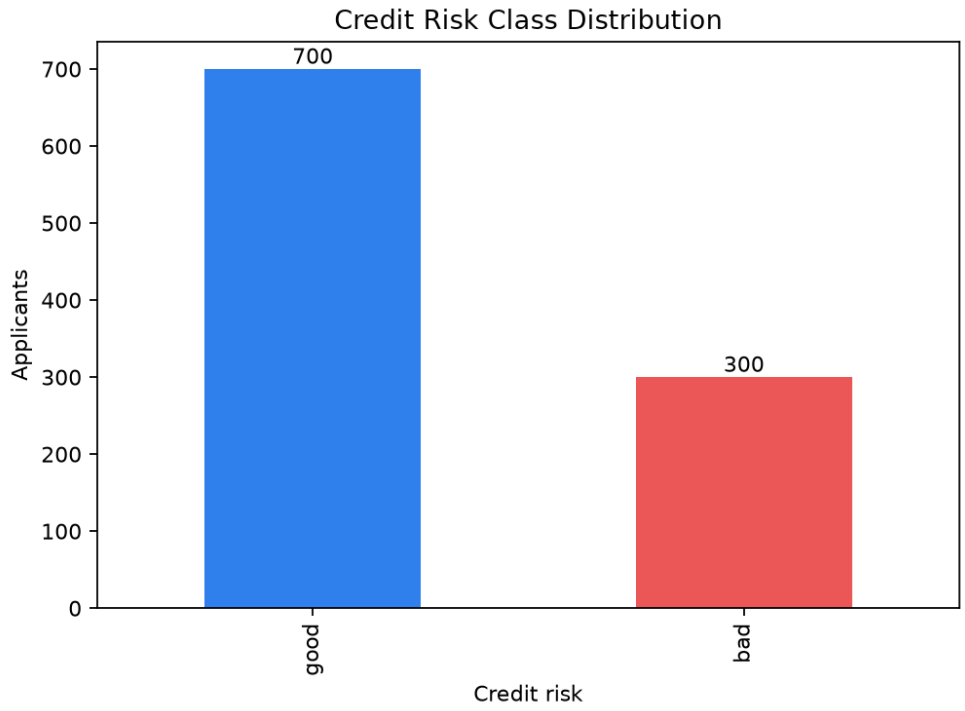


Fig. 6.1 Observed good-risk and bad-risk target distribution

6.5 Model Training

The experiment compares LR, a depth-five DT, RF and GB. LR, RF and GB use eight-candidate randomized hyperparameter searches scored by five-fold stratified ROC-AUC. DT remains an untuned interpretable baseline. Training seeds are fixed. The selected GB uses 200 estimators, learning rate 0.05, depth 3, minimum leaf size 5 and subsample 0.8.

6.6 Model Comparison

The comparison reports accuracy, precision, bad-risk recall, F1, ROC-AUC, confusion matrices and cost. It preserves both conventional-threshold and training-selected-threshold results. LR has the highest primary holdout ROC-AUC, 0.8095; GB has 0.8005. Model ranking depends on the metric and operating point, so the report does not describe one classifier as universally best.

Table 6.1 Independent test performance at training-selected thresholds

Model	Acc.	Prec.	Recall	F1	AUC	Cost
LR	0.6050	0.4240	0.8833	0.5730	0.8095	107
DT	0.6050	0.4159	0.7833	0.5434	0.6741	131
RF	0.5300	0.3836	0.9333	0.5437	0.7912	110
GB	0.5500	0.3881	0.8667	0.5361	0.8005	122

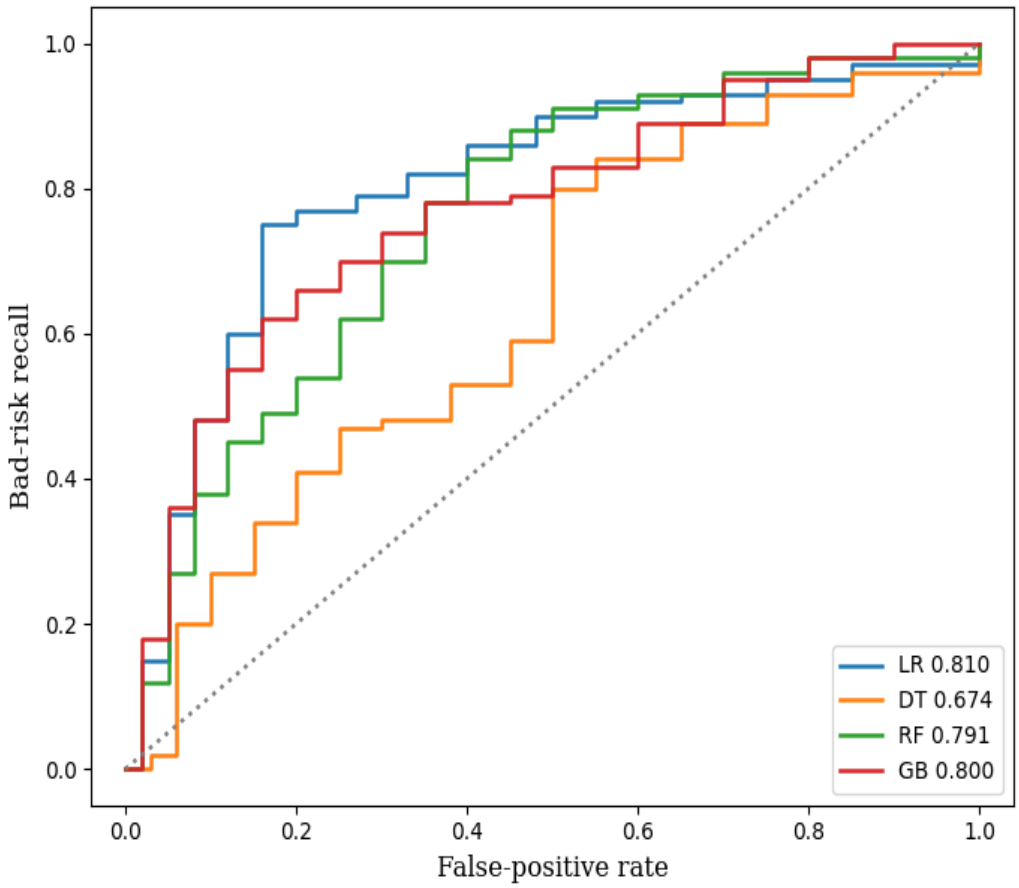


Fig. 6.2 Four-model ROC curves on the independent holdout

6.7 Cost-Sensitive Evaluation

The implemented cost is C = 5FN + FP. A false negative is an actual bad-risk applicant predicted good; a false positive is an actual good-risk applicant predicted bad. These are relative benchmark penalties, not monetary estimates. For selected GB, eight false negatives and 82 false positives produce cost 5 × 8 + 82 = 122.

6.8 Decision-Threshold Selection

Each model receives five-fold OOF probabilities on the 800 development rows. The threshold search covers 0.10–0.90 in steps of 0.01. Minimum training cost wins, with recall and F1 resolving ties. LR, DT, RF and GB receive thresholds 0.18, 0.10, 0.34 and 0.13. GB has the lowest training cost, 398, and is saved without using holdout outcomes to reverse the choice.

Table 6.2 Training-selected thresholds and costs

Model	Threshold	OOF cost	Holdout cost
LR	0.18	410	107

Model	Threshold	OOF cost	Holdout cost
DT	0.1	476	131
RF	0.34	416	110
GB	0.13	398	122

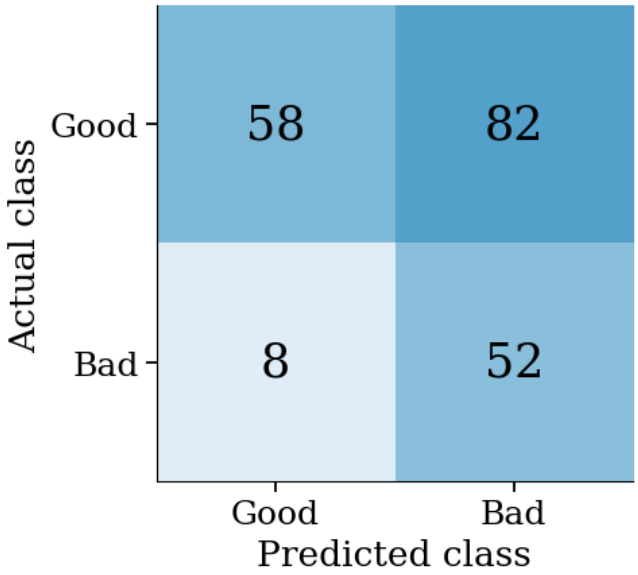


Fig. 6.3 Selected GB confusion matrix at threshold 0.13

6.9 Risk Probability and Risk Score

Inference locates class label 1 in the fitted classes before extracting predict_proba. The score is 100 times that model output, with finite-range validation. LOW is below 30, MODERATE is 30 to below 60, and HIGH is 60 through 100. These project-defined bands organize educational presentation; they do not determine loan approval or establish a calibrated probability of default.

6.10 SHAP Explainability

Tree SHAP is computed after applying the saved transformer. Additivity is checked against the GB decision margin, and its logistic transformation is checked against class-1 probability. Positive SHAP values increase modeled bad risk and negative values decrease it. Publication global importance uses only 200 holdout applicants. The existing dashboard global artifact was generated over all 1,000 records and is distinguished from this holdout-only publication analysis.

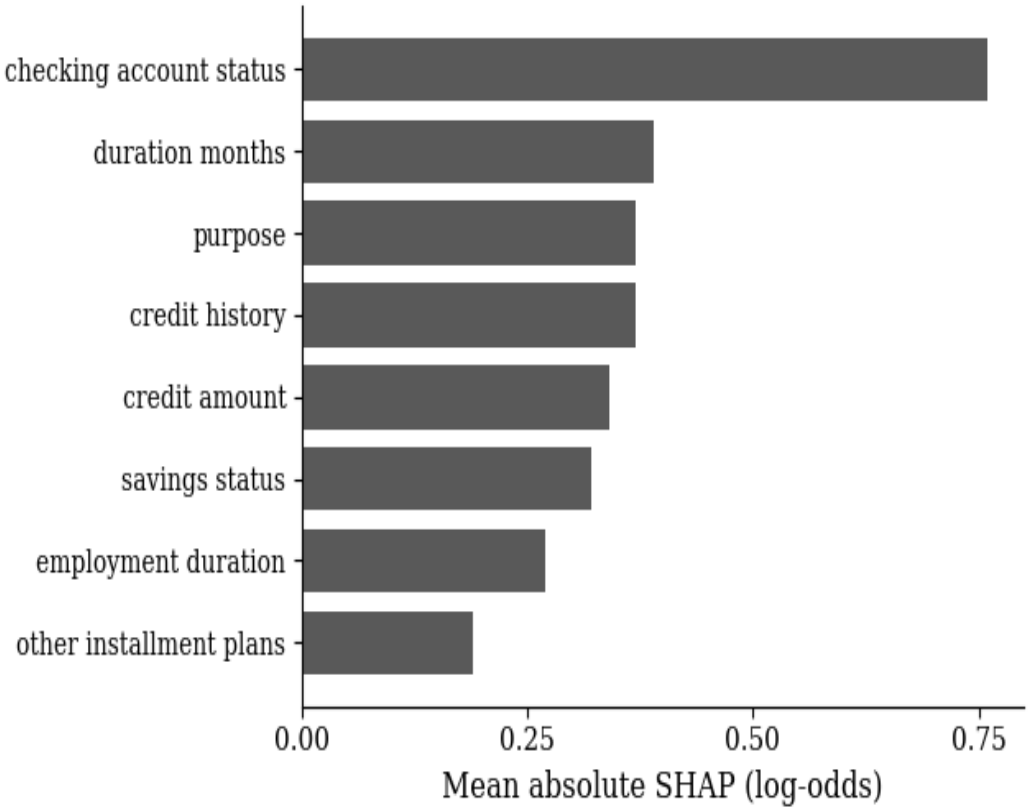


Fig. 6.4 Global SHAP importance calculated only on the 200 holdout applicants

6.11 Flask Backend

dashboard/app.py provides the thin request layer. It loads the artifact, metadata, processed analytics and evaluation outputs, validates requests and calls reusable inference and explanation modules. It rejects identifiers and outcome fields in applicant submissions. Automated guards verify that request handling does not fit a model or transformer.

6.12 Dashboard Front-End

Jinja templates provide overview, portfolio, assessment, explainability, performance and What-If pages. Bootstrap and local CSS structure the layout. Plotly renders portfolio charts from real processed data. Model evaluation views read generated metrics; they do not perform new searches when opened. Some browser assets are supplied by public CDNs and need network access for full visual behavior.

6.13 What-If Simulator

The simulator validates an original and an updated profile, predicts each with the same saved pipeline and reports probability and score differences. Identical inputs produce no change. A

changed result represents model sensitivity to the supplied inputs and is not evidence that changing an applicant attribute will cause an equivalent change in real repayment risk.

6.14 AWS S3 Integration

The cloud helpers support upload_file, download_file and object_exists through Boto3 credential resolution. A 403 authorization response is not treated as a missing object. Credentials are configured outside source code. The deployed analytics CSV and raw data are held in S3; cloud access remains optional for local dashboard execution.

6.15 AWS Glue and Athena

The analytics CSV is described by an external table in the finsight Glue Data Catalog database. Athena DDL registered the table without a crawler. Reusable SQL covers applicant counts, target counts, average credit amount, average duration and grouped risk summaries. The region recorded by the project is eu-north-1. Setup permissions and resource locations are configuration concerns, not machine-learning novelty.

CHAPTER 7
FEATURES

Table 7.1 Feature routes and data sources

Feature	Route / execution	Source
Overview and portfolio	/ and /portfolio	Processed analytics CSV
Applicant assessment	/predict	Saved pipeline and input profile
Explanations	/explain	Saved global plot and local session factors
Performance	/performance	Model metrics and saved plots
What-If comparison	/whatif	Two profiles; same pipeline
Cloud analytics	Athena console / SQL	S3 CSV and Glue catalog

7.1 Portfolio Analytics

Users inspect applicant counts, risk distribution and grouped relationships. Pandas reads the processed CSV and Plotly creates interactive charts. These summaries describe benchmark records rather than a live lending portfolio.

7.2 Applicant Risk Prediction

Users enter 20 raw predictors using validated numeric controls and categorical selections. The server applies the saved pipeline and threshold, returning a good/bad model class with supporting factors.

7.3 Risk Probability and Educational Score

The result card shows the model's class-1 output and its 0–100 scaled score. The computation is explicit and does not imply that the number is an official credit score or a proven probability of default.

7.4 Low, Moderate and High Bands

A badge groups the educational score using boundaries 30 and 60. Neutral actions are Lower Predicted Risk, Manual Review Suggested and Higher Predicted Risk. Band labels remain separate from the binary threshold.

7.5 SHAP Explainability

Users see signed risk-increasing and protective factors for the assessed applicant. The explanation page also shows the pre-generated global artifact. SHAP describes the fitted model; it does not certify fairness or establish causal effects.

7.6 Model Performance and Asymmetric Cost

Users compare the four classifiers, saved ROC curves and confusion matrix. Metrics are loaded from generated JSON rather than calculated from invented demonstration values. Cost explicitly penalizes missed bad-risk applicants five times as strongly as false alarms.

7.7 What-If Simulator

Users provide original and updated valid profiles. The same estimator generates both predictions and the interface reports their difference. This supports sensitivity exploration while retaining an explicit non-causal interpretation.

7.8 AWS Analytics

A configured AWS user can query the S3 analytics CSV through Athena using Glue metadata. The local Flask pages continue to work without a live AWS connection; no cloud hosting of the application is claimed.

CHAPTER 8
TESTING

8.1 Test Strategy

The repository test suite exercises data loading and validation, preprocessing, model loading, inference, probability orientation, score boundaries, SHAP consistency, cloud helpers with mocks, and Flask request behavior. HTTP tests use Flask's test client. They verify application integration without requiring a public server or live AWS credentials. Mocked AWS tests are not evidence of newly executed live-cloud operations.

8.2 Verified Execution

The complete command .venv\Scripts\python.exe -m pytest -q was executed during this report task. Result: 81 passed, 0 failed, 50 upstream warnings, elapsed time 93.79 seconds. Warnings concern SHAP/Matplotlib and Joblib/NumPy deprecations. Table 8.1 groups representative covered behaviors; rows summarize related assertions rather than claiming there are only eleven individual tests.

Table 8.1 Representative test-case coverage

ID	Module	Description	Expected	Actual	Status
TC01	Data loading	Read official cached source	1,000 × 21 raw schema	Verified by passing suite	PASS
TC02	Validation	Check values, codes and integrity	Valid source accepted; invalid data rejected	Verified by passing suite	PASS
TC03	Preprocessing	Split and transform without leakage	80/20 stratification; training-fitted transforms	Verified by passing suite	PASS
TC04	Artifact	Load saved model and feature contract	GB pipeline with threshold 0.13	Verified by passing suite	PASS
TC05	Prediction	Evaluate valid and invalid profiles	Valid output; invalid inputs rejected	Verified by passing suite	PASS
TC06	Orientation	Resolve positive probability by class label	Class 1 consistently means bad risk	Verified by passing suite	PASS
TC07	Score	Check score range and band boundaries	Finite 0–100 score; correct 30/60 boundaries	Verified by passing suite	PASS
TC08	Threshold	Apply saved operating cutoff	Binary class matches saved threshold	Verified by passing suite	PASS
TC09	SHAP	Check additivity and factor directions	Verified model-margin consistency	Verified by passing suite	PASS

ID	Module	Description	Expected	Actual	Status
TC10	Flask	Exercise six pages and form responses	Successful routes and validation messages	Verified by passing suite	PASS
TC11	Integration	Check cloud mocks and no-fit guards	Expected helpers; no training during requests	Verified by passing suite	PASS

8.3 Research Verification

Publication analysis reproduced the four saved operating thresholds and holdout metrics exactly before adding 2,000 bootstrap resamples, six exact McNemar tests with Holm correction, training-only sigmoid calibration and ten development-only validation folds. These are additional research analyses, separate from the 81 project tests. Original tuning is not fully nested, and the repeat experiment uses prespecified untuned classifiers with inner threshold selection.

CHAPTER 9
DEPLOYMENT

9.1 Local Setup

Use a local checkout containing dashboard, src, tests, data, models and generated outputs. Run commands from the project root. The verified environment is Windows with PowerShell. A compatible Python interpreter and browser are required.

9.2 Virtual Environment

Create .venv only when absent using python -m venv .venv. On PowerShell, the explicit .venv\Scripts\python.exe path avoids activation-policy issues. Keep the environment outside version control.

9.3 Dependency Installation

Install the declared requirements with .venv\Scripts\python.exe -m pip install -r requirements.txt. The package bounds in requirements.txt define the application dependencies. Retain compatible package versions for loading existing serialized artifacts.

9.4 Model Training

Run the data command and EDA/preprocessing commands before .venv\Scripts\python.exe -m src.models.train when intentionally regenerating training artifacts. Training refreshes the saved pipeline, metadata and evaluation outputs. Routine dashboard startup should load the existing artifact rather than retrain.

9.5 Flask Execution

Start .venv\Scripts\python.exe -m dashboard.app and open http://127.0.0.1:5000 in a browser. The existing development entry point binds to localhost with debug disabled. This is local execution; a public production deployment is not claimed.

9.6 AWS S3 Setup

Configure an AWS profile or role outside the repository. The project's deployed analytics location is documented in the AWS setup report. A new environment requires a private bucket/prefix and appropriate read/write permissions. No account identifiers or credentials are embedded in this report.

9.7 AWS Glue

Register the processed CSV schema in Glue Data Catalog. The implemented route uses Athena CREATE EXTERNAL TABLE DDL; crawler creation was unavailable to the configured identity. Crawler provisioning is optional and must not be described as completed.

9.8 Amazon Athena

Configure an Athena query-result location and execute the generated SQL against the external table. Validate counts and grouped risk totals against local outputs. The stored project record identifies database finsight and table german_credit_analytics.

9.9 Environment Configuration

Use AWS_PROFILE and AWS_DEFAULT_REGION for external AWS configuration where needed. The query helper supports FINSIGHT_ATHENA_DATABASE and FINSIGHT_ATHENA_TABLE overrides. Flask session configuration should be supplied securely for any future deployment. Never commit credentials, private keys or environment secrets.

CHAPTER 10
CHALLENGES AND LIMITATIONS

Table 10.1 Challenges, implemented responses and remaining limits

Challenge	Implemented response	Remaining limitation
70:30 class imbalance	Report bad-risk recall, F1, AUC and cost	Historical prevalence may not represent deployment
Unequal error costs	OOF threshold selection under 5FN + FP	Fixed ratio omits applicant-specific exposure
Target leakage	Raw-schema contract and pipeline preprocessing	Original hyperparameter evaluation is not fully nested
High recall versus false alarms	Explicit thresholds and confusion matrices	GB flags 82 good-risk holdout applicants
Probability quality	Training-only sigmoid calibration evaluated	Brier score did not improve
Model generalization	Independent holdout and repeated development analysis	Small historical benchmark; no external validation
Explanation reliability	SHAP additivity and probability-direction checks	No causality, stability or fairness guarantee
AWS permissions	Catalog table registered through Athena DDL	Crawler provisioning unavailable; setup is account-specific
Local application	Saved-model Flask execution and input checks	No production hosting, authentication or monitoring

10.1 Threshold and Generalization Trade-Offs

At threshold 0.13, GB recall rises from 0.5500 to 0.8667 compared with 0.50, while accuracy falls from 0.7900 to 0.5500 and precision falls from 0.6875 to 0.3881. Cost falls from 150 to 122 because the reduction in missed bad-risk applicants outweighs the additional false alarms under the selected ratio. LR nevertheless has lower primary holdout cost, 107. The observed reversal limits confidence in choosing a universal winner from development estimates.

10.2 Calibration, Explanations and Fairness

Sigmoid calibration changes holdout Brier score from 0.1562 to 0.1569; its own training-selected threshold 0.20 gives cost 106 and AUC 0.7993. Better operating cost does not imply improved probability calibration. SHAP explains model associations in log-odds and does not establish why people repay or default. Sensitive/proxy attributes and historical coding issues require a separate fairness and domain review before any operational consideration.

10.3 Data and Evaluation Limitations

The original German Credit coding documentation contains errors; UCI's corrected South German Credit release [14] addresses the same historical source rather than furnishing an independent contemporary cohort. Its download was unavailable during publication preparation, so no corrected-data result is asserted. Bootstrap intervals condition on fixed fits, McNemar tests compare correctness rather than cost, and repeated validation of untuned defaults does not replace fully nested tuning. Explanation-stability studies [10], [11] motivate further checks.

CHAPTER 11
FUTURE ENHANCEMENTS

11.1 Data and Validation

Validate the corrected South German representation separately after auditing its coding, and evaluate an additional credit-default dataset such as the Taiwan benchmark with an appropriate feature and target contract. Larger, more recent and temporally separated institution data are needed to assess transfer. Fully nested evaluation should include preprocessing, search, calibration, threshold selection and model selection inside each outer training partition.

11.2 Statistical and Responsible-ML Evaluation

Bootstrap intervals, Holm-corrected McNemar tests, a separate repeated-validation protocol and sigmoid calibration have already been completed in the publication extension and are not presented here as untouched future work. Future evaluation should include retraining uncertainty, paired cost comparisons, alternative calibration strategies, subgroup fairness auditing and explanation stability under resampling or drift. Existing intervals and tests should be repeated on new independent datasets.

11.3 Application and Cloud Extensions

Future engineering work includes authentication, role-based access, audit logging, privacy controls, model monitoring and controlled model-version changes. A secure cloud deployment would require an appropriate production WSGI server, TLS, external secrets management and restricted AWS access. These are proposals rather than features of the current local educational prototype.

CHAPTER 12
CONCLUSION

12.1 Summary

FinSight integrates a validated benchmark, leakage-controlled preprocessing, four-classifier evaluation, asymmetric threshold selection, saved-model inference, SHAP explanations, a local Flask dashboard and optional AWS analytics. The project connects experimental results to the interface through one explicit feature, target and threshold contract.

12.2 Achievements

Implemented outcomes include reproducible data acquisition and reports, model and metadata artifacts, portfolio and applicant workflows, signed local explanations, What-If sensitivity comparisons and a verified S3/Glue/Athena analytics path. All 81 current project tests pass. Publication additions reproduce the saved results and provide holdout-only global SHAP, threshold diagnostics, uncertainty intervals, statistical comparisons and calibration assessment.

12.3 Actual Model Findings

GB was selected from training evidence at threshold 0.13 with OOF cost 398. Its holdout accuracy is 0.5500, precision 0.3881, recall 0.8667, F1 0.5361, ROC-AUC 0.8005 and cost 122. LR has the lowest primary holdout cost, 107. Among six paired correctness tests, only LR–RF remains significant after Holm correction (adjusted $p = 0.0488$). These findings support careful operating-point analysis rather than a claim that GB is always superior.

12.4 Learning Outcomes

The work demonstrates how data contracts, train/test separation, probability orientation, cost assumptions, uncertainty and explainability affect an end-to-end application. We learned to distinguish descriptive analytics from model evaluation, educational scores from official credit measures, and model sensitivity from causality. FinSight is an educational decision-support prototype and cannot autonomously approve or reject real loans.

CHAPTER 13
REFERENCES

[1] N. Bussmann, P. Giudici, D. Marinelli, and J. Papenbrock, “Explainable AI in Fintech Risk Management,” Frontiers in Artificial Intelligence, vol. 3, Art. no. 26, 2020, doi: 10.3389/frai.2020.00026.

[2] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” Nature Machine Intelligence, vol. 1, no. 5, pp. 206–215, 2019, doi: 10.1038/s42256-019-0048-x.

[3] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in Advances in Neural Information Processing Systems, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions (no publisher DOI assigned).

[4] X. Dastile and T. Celik, “Making Deep Learning-Based Predictions for Credit Scoring Explainable,” IEEE Access, vol. 9, pp. 50426–50440, 2021, doi: 10.1109/ACCESS.2021.3068854.

[5] M. Bücker, G. Szepannek, A. Gosiewska, and P. Biecek, “Transparency, auditability, and explainability of machine learning models in credit scoring,” Journal of the Operational Research Society, vol. 73, no. 1, pp. 70–90, 2022, doi: 10.1080/01605682.2021.1922098.

[6] P. E. de Lange, B. Melsom, C. B. Vennerød, and S. Westgaard, “Explainable AI for Credit Assessment in Banks,” Journal of Risk and Financial Management, vol. 15, no. 12, Art. no. 556, 2022, doi: 10.3390/jrfm15120556.

[7] M. K. Nallakaruppan, B. Balusamy, M. Lawanya Shri, V. Malathi, and S. Bhattacharyya, “An Explainable AI framework for credit evaluation and analysis,” Applied Soft Computing, vol. 153, Art. no. 111307, 2024, doi: 10.1016/j.asoc.2024.111307.

[8] A. Gramegna and P. Giudici, “SHAP and LIME: An Evaluation of Discriminative Power in Credit Risk,” Frontiers in Artificial Intelligence, vol. 4, Art. no. 752558, 2021, doi: 10.3389/frai.2021.752558.

[9] J. Černevičienė and A. Kabašinskas, “Explainable artificial intelligence (XAI) in finance: a systematic literature review,” Artificial Intelligence Review, vol. 57, no. 8, Art. no. 216, 2024, doi: 10.1007/s10462-024-10854-8.

[10] M. Ballegeer, M. Bogaert, and D. F. Benoit, “Evaluating the stability of model explanations in instance-dependent cost-sensitive credit scoring,” European Journal of Operational Research, vol. 326, no. 3, pp. 630–640, 2025, doi: 10.1016/j.ejor.2025.05.039.

[11] Y. Chen, R. Calabrese, and B. Martin-Barragan, “Interpretable machine learning for imbalanced credit scoring datasets,” European Journal of Operational Research, vol. 312, no. 1, pp. 357–372, 2024, doi: 10.1016/j.ejor.2023.06.036.

[12] Y. Li, F. Gao, M. Sha, and X. Shao, “Sequential three-way decision with automatic threshold learning for credit risk prediction,” Applied Soft Computing, vol. 165, Art. no. 112127, 2024, doi: 10.1016/j.asoc.2024.112127.

[13] H. Hofmann, “Statlog (German Credit Data),” UCI Machine Learning Repository, dataset, 1994, doi: 10.24432/C5NC77.

[14] “South German Credit,” UCI Machine Learning Repository, dataset, 2020, doi: 10.24432/C5QG88.

[15] Scikit-learn developers, “Scikit-learn documentation.” [Online]. Available: https://scikit-learn.org/stable/. Accessed: Nov. 5, 2026.

[16] Pallets, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com/en/stable/. Accessed: Nov. 5, 2026.

[17] Amazon Web Services, “What is Amazon S3?.” [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html. Accessed: Nov. 5, 2026.

[18] Amazon Web Services, “Data discovery and cataloging in AWS Glue.” [Online]. Available: https://docs.aws.amazon.com/glue/latest/dg/catalog-and-crawler.html. Accessed: Nov. 5, 2026.

[19] Amazon Web Services, “What is Amazon Athena?.” [Online]. Available: https://docs.aws.amazon.com/athena/latest/ug/what-is.html. Accessed: Nov. 5, 2026.

CHAPTER 14

APPENDICES

14.1 Architecture and Workflow

The architecture and workflow are shown in Figs. 4.1, 5.1 and 5.2. They distinguish local development and inference from the optional cloud analytics path. No application database or entity–relationship model is implied.

14.2 Dashboard Screenshots

The following figures were captured from the actual local Flask application. They show implemented pages and model-generated outputs, rather than mockups. The global SHAP image displayed by the dashboard is the existing project artifact; Fig. 6.4 separately provides the corrected holdout-only publication analysis.

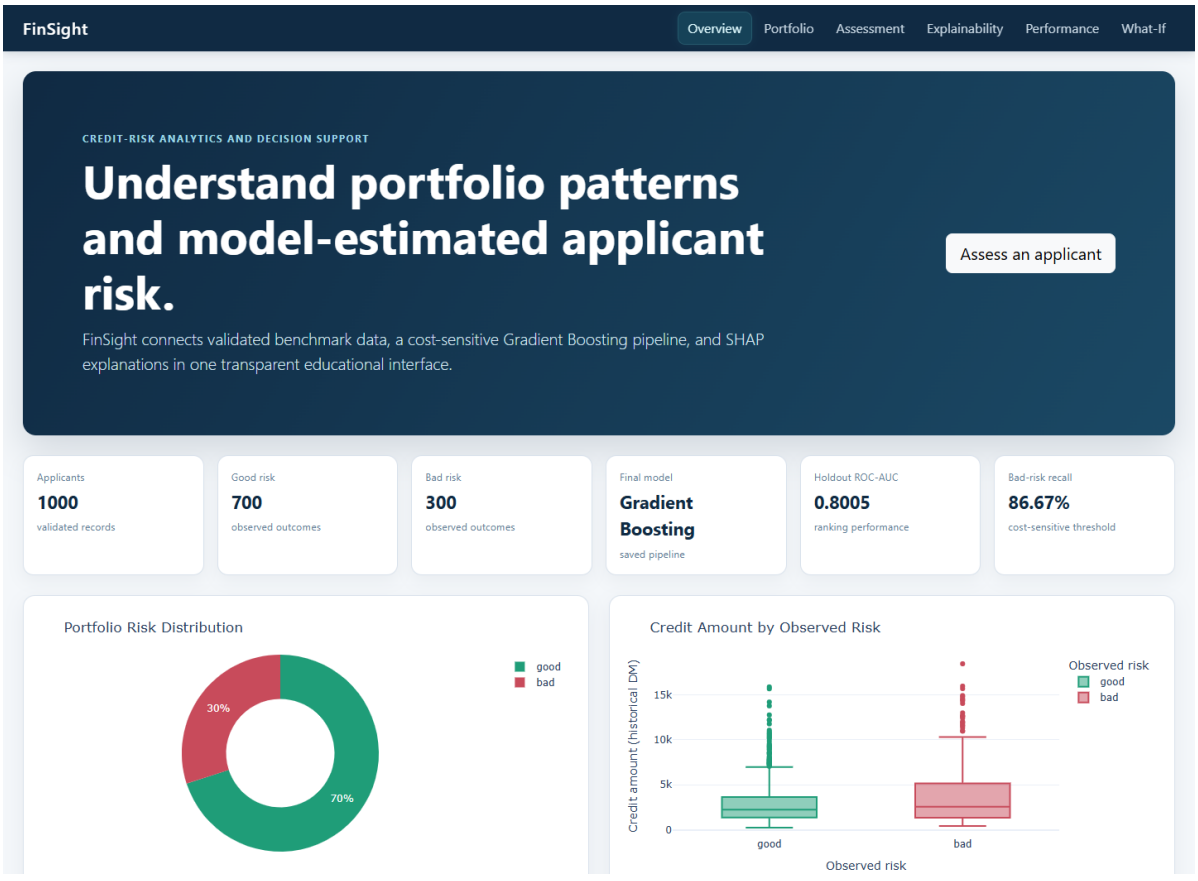


Fig. 14.1 Dashboard Overview

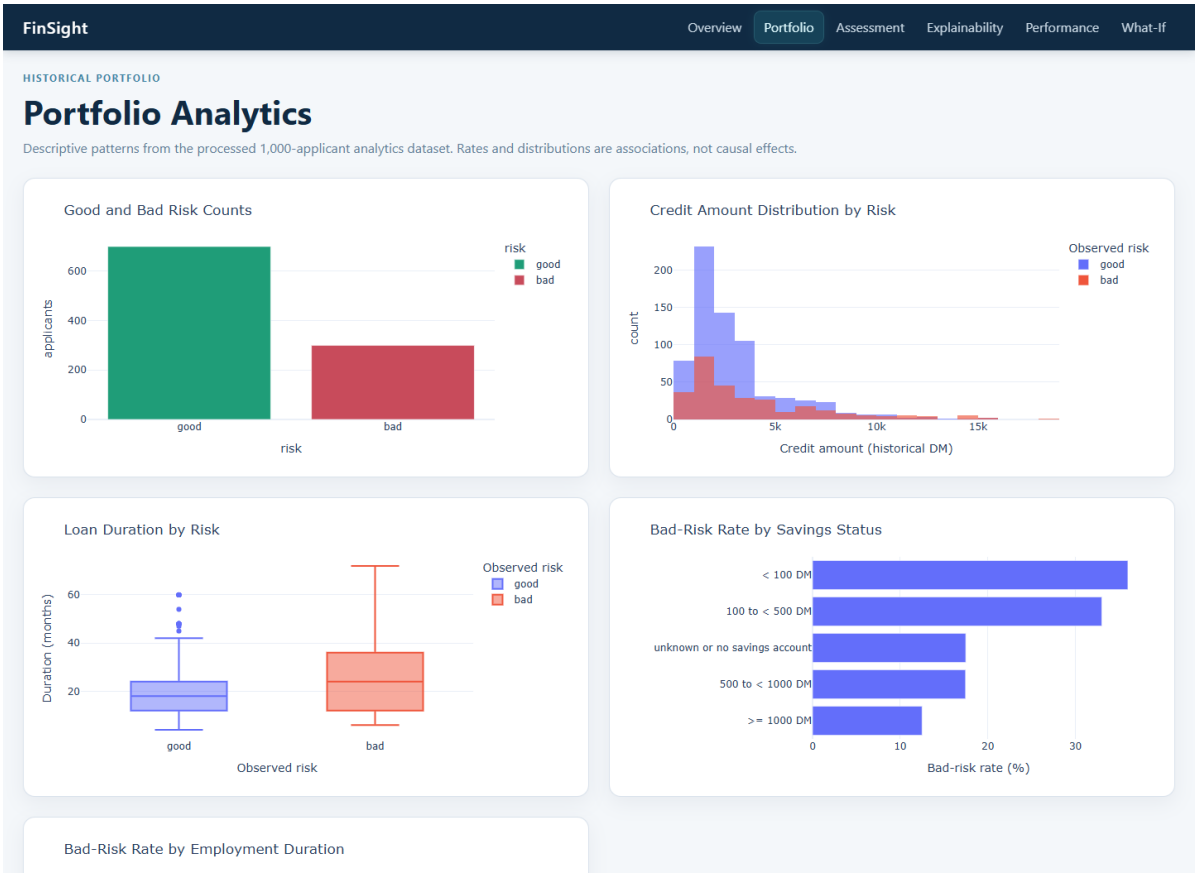


Fig. 14.2 Portfolio Analytics

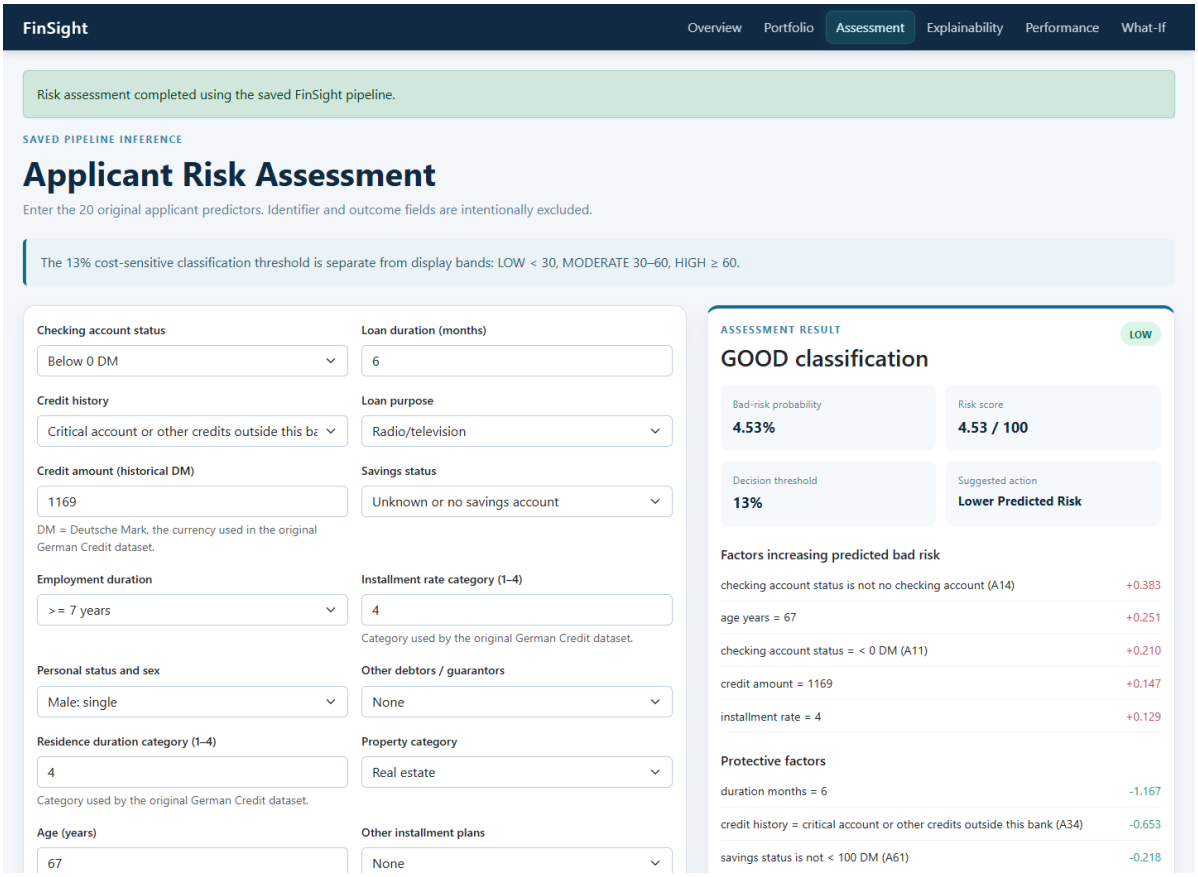


Fig. 14.3 Applicant Risk Prediction

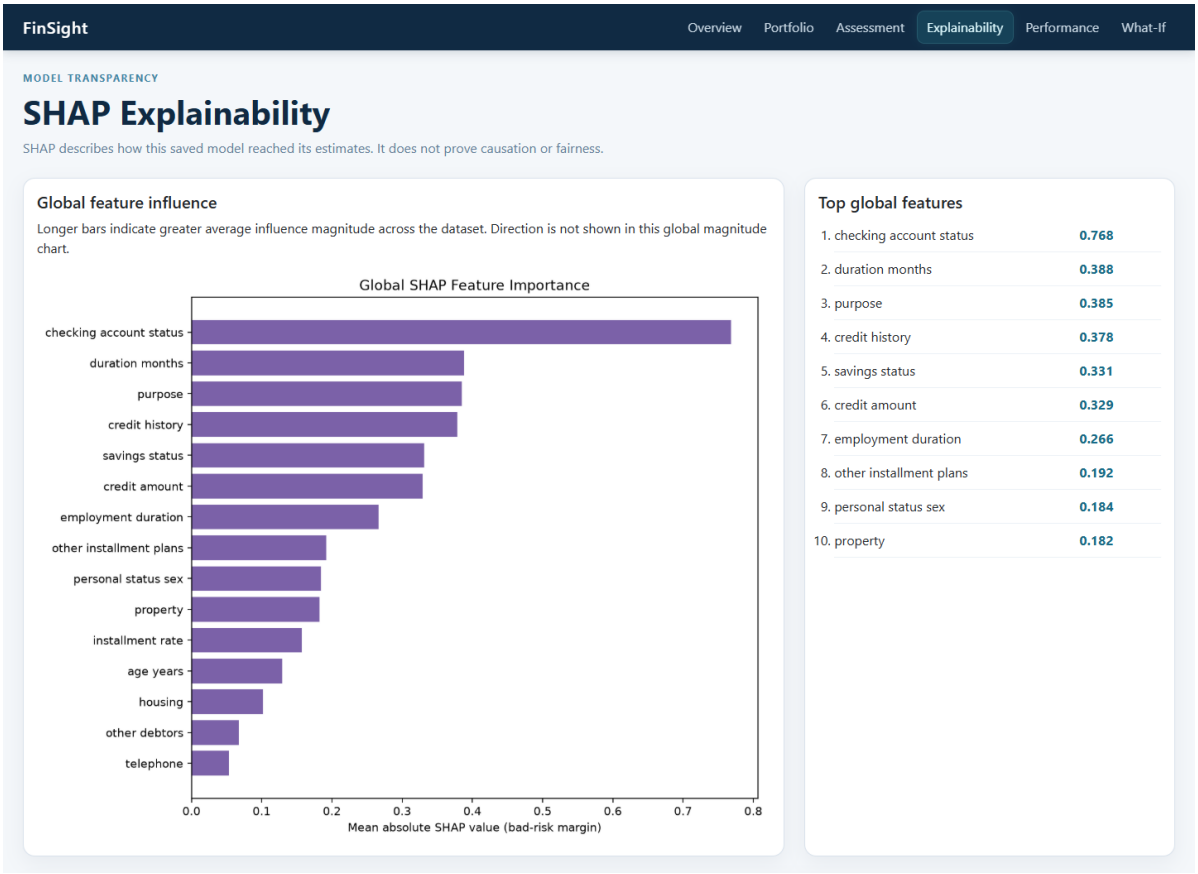


Fig. 14.4 SHAP Explainability

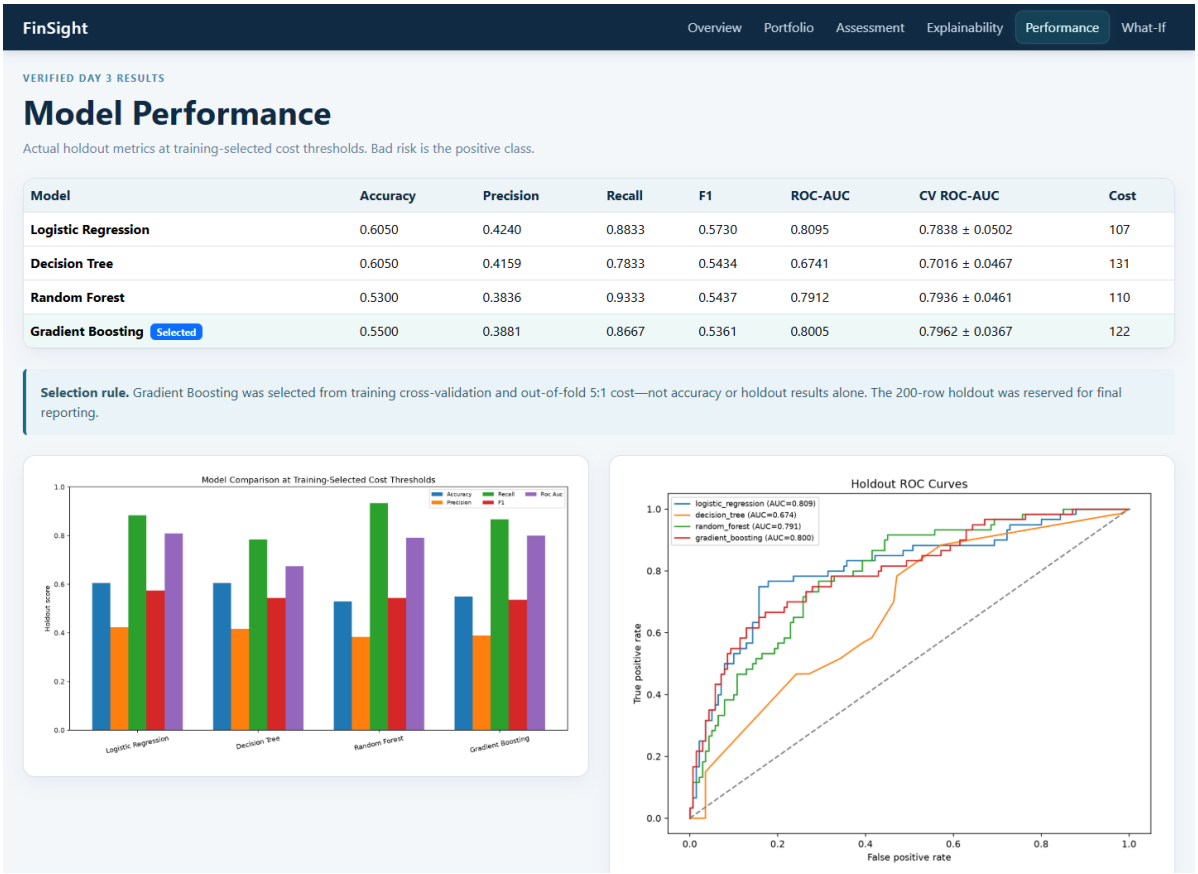


Fig. 14.5 Model Performance

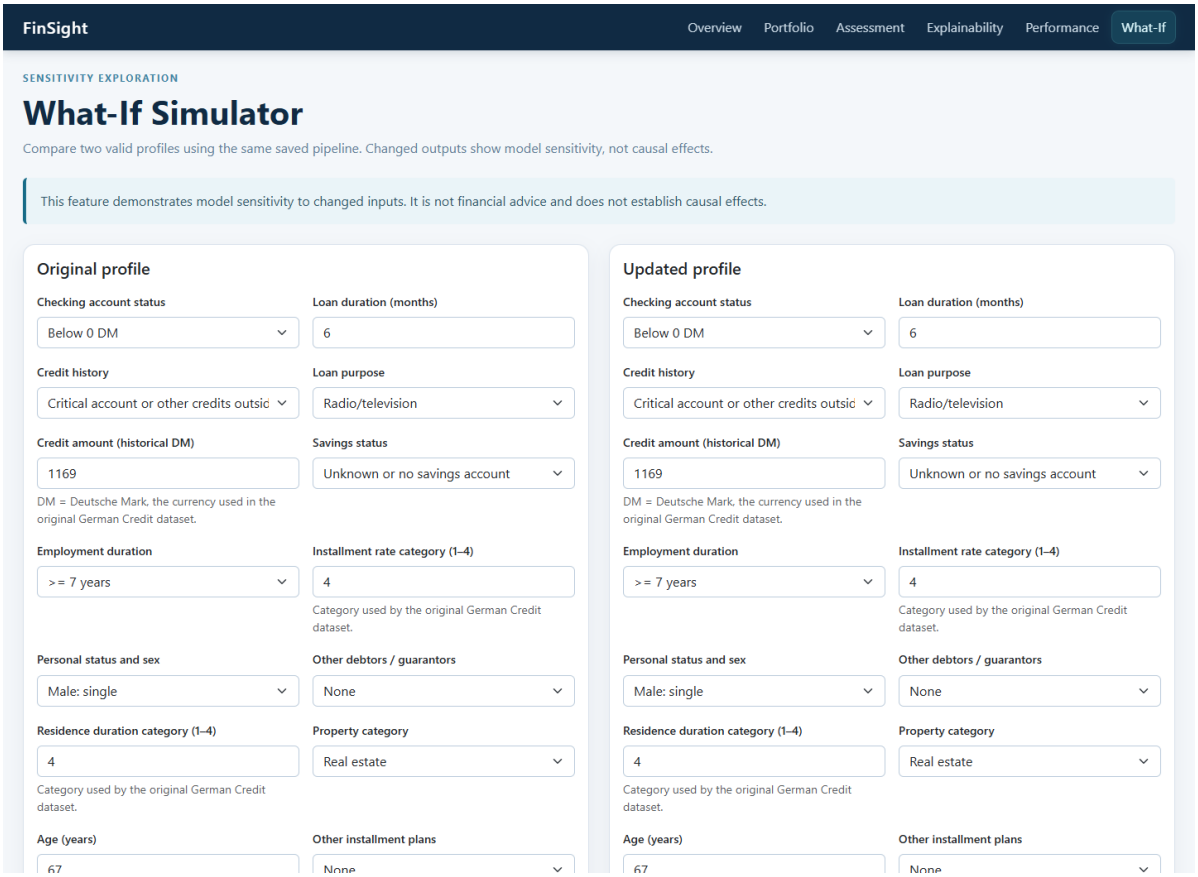


Fig. 14.6 What-If Simulator

14.3 Additional Evaluation Evidence

Figs. 6.2 and 6.3 provide the ROC and selected-model confusion matrix. Fig. 14.7 shows threshold sensitivity using the implemented grid. The holdout curves are diagnostic and do not select a new operational cutoff.

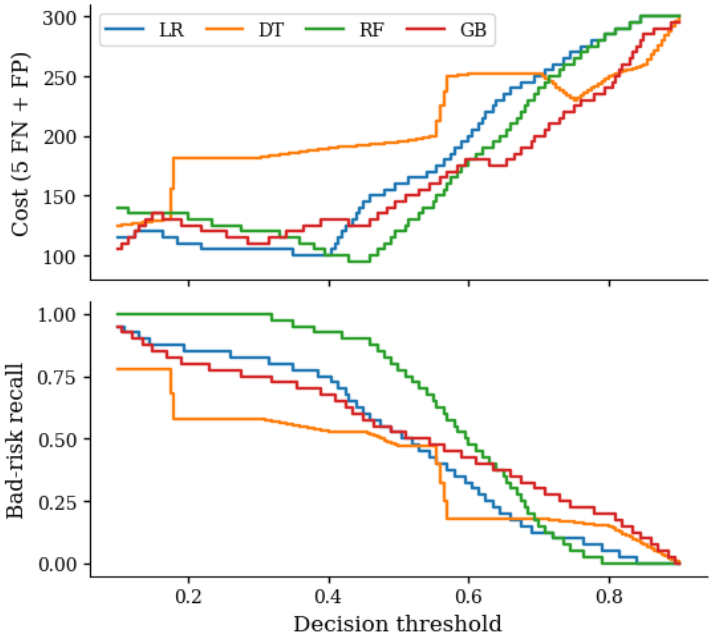


Fig. 14.7 Holdout cost and recall across decision thresholds

Table 14.1 Selected-model 95% bootstrap intervals

Metric	Estimate	95% interval
accuracy	0.55	0.4800 to 0.6200
precision	0.3881	0.3060 to 0.4710
recall	0.8667	0.7736 to 0.9474
f1	0.5361	0.4457 to 0.6180
roc_auc	0.8005	0.7233 to 0.8677
cost	122	95.0000 to 152.0000

14.4 Selected Code Snippets

The following excerpts are selected from the implementation to illustrate the target orientation, score definition and cost contract. Full source code remains in the repository; no unrelated sample code is retained.

```
classes = np.asarray(model.classes_)
bad_index = int(np.flatnonzero(classes == 1)[0])
bad_risk_probability = pipeline.predict_proba(X)[:, bad_index]
```

```
risk_score = bad_risk_probability * 100.0
prediction = int(bad_risk_probability >= artifact["threshold"])
total_cost = 5 * false_negatives + false_positives
```

The snippets summarize the existing inference, score and experiment functions. The class-label lookup is essential: blindly assuming the meaning of a probability column can reverse the interpretation of risk.

14.5 Installation and Run Commands

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m src.data.run_day1
.venv\Scripts\python.exe -m src.data.run_eda
.venv\Scripts\python.exe -m src.data.preprocess
.venv\Scripts\python.exe -m src.models.train
.venv\Scripts\python.exe -m src.models.run_day4
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m dashboard.app
```

Create the virtual environment only when it is absent. Training and Day 4 generation intentionally refresh their respective artifacts; they are not required for every dashboard launch when valid saved outputs already exist. Use external AWS identity configuration for optional cloud operations.

14.6 User Guide

Open the local dashboard and inspect Portfolio for descriptive context. On Assessment, complete all 20 predictors using the displayed category meanings and historical units. Read class, model output, score and signed factors together. Open Explainability to inspect the current applicant's factors and the saved global figure. Use Performance to compare saved model evidence. On What-If, submit original and updated profiles, then interpret the output difference as model sensitivity only. Invalid or missing fields must be corrected before assessment.

14.7 Test Evidence

Complete suite: 81 passed, 0 failed, 50 warnings in 93.79 seconds. The test files are test_data.py, test_day1.py, test_day2.py, test_preprocessing.py, test_model_inference.py, test_day4.py and test_dashboard.py. The publication extension separately records all four model metrics, 2,000-resample bootstrap intervals, exact paired tests, calibration and repeated-validation fold outputs. No new live AWS verification is claimed by the local test run.